2026 年第八届
全球校园人工智能算法精英大赛

算法主题赛
AI+ 开源
技术报告

团队名称：应光

团队编号：AIC-2026-88229444

作品名称：光语者——面向数字无障碍的桌面智能体

参赛方向：开源赋能的 AI 应用创新

日期：2026 年 10 月 26 日

目录

一、作品概述 1

二、需求分析与总体方案 1

三、AI 技术与开源及第三方资源选择 2

四、系统设计与实现 3

五、真实场景验证与成果展示 9

六、创新性与开放价值 12

七、安全、隐私与合规 14

八、总结与展望 15

九、附录 16

一、作品概述

为长辈安排出行，往往需要在网站比较车次、核对接驳条件，再到微信发送信息。政务办事、就医准备和购物售后也有类似特点：用户目标明确，操作却分散在多个网站和应用中，对老年人及不熟悉数字工具的用户尤其繁琐。

多模态模型已经能够理解界面，开源 Computer-Use 工具也提供了桌面输入能力；实际任务仍需要处理窗口变化、结果核对、用户临时修改要求与关键操作确认。**光语者（Lightspeaker）通过手机文字或语音接受任务，由用户自己的电脑执行，并让用户随时查看、纠正和确认。**大字、读屏支持、语音播报及个性化配置构成其面向不同使用者的交互入口。

项目基于开源 CUA，自主构建 **Computer Harness** 运行时，组织模型决策、跨应用状态、任务记忆、执行核对与人机控制。作品申报“开源赋能的 AI 应用创新”，融合“开源项目改进与生态贡献”。团队围绕六类生活场景开展真实任务测试，并通过 Windows 输入问题的复现和修复与 CUA 维护者协作，另向 OpenClaw 贡献两项已合并改进。

二、需求分析与总体方案

2.1 从操作界面回到生活目标

生活任务的困难不仅在单次点击，也在条件持续有效：取消预约需要核对方式和时限，出行比较需要同时满足日期、车站和票价要求，交付时还要确认接收对象。对老年人和存在阅读障碍的使用者，小字、分散的状态提示及不一致的应用操作进一步增加了负担。

因此，Lightspeaker 以完整事务为单位，让用户描述目标、补充条件并了解进度。普通查询和草稿整理应顺畅推进，发送、提交等关键操作则清楚说明后果并请求确认；用户不必为了审批反复返回电脑，也无需理解坐标和窗口句柄。

2.2 系统整体形态

系统由手机 Web、Relay 中继、电脑 Host 与 Computer Harness 构成。手机负责目标输入、状态查看和用户决定；Relay 连接两端并转发任务消息；Host 管理本机模型配置、桌面连接与受管浏览器状态；Harness 运行模型决策与桌面执行循环。电脑主动连接 Relay，手机通过配对获得相应任务的控制入口。

用户发起任务后，Host 固定模块开关、预算与操作范围，创建运行实例。模型调用、执行回执、后续观察以及用户纠正都属于该实例。专用受管浏览器复用准备好的登录状态，微信、编辑器等原生应用通过桌面能力参与任务；手机不保存模型密钥或原生窗口句柄。

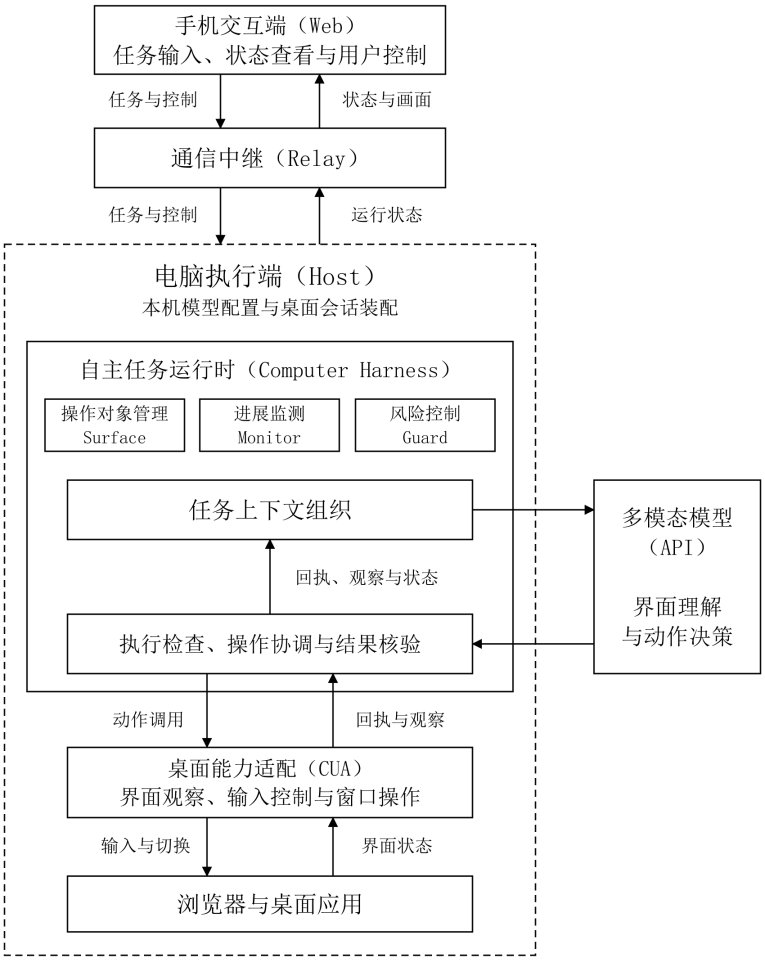


图 1　Lightspeaker 的交互、任务运行与桌面执行关系。手机操作与模型决策进入同一个任务控制层；底层桌面能力由 CUA 及其平台适配提供。

2.3 实施可行性与成本

项目调用 GLM、Qwen 等云端模型，利用 CUA[1] 提供桌面操作能力，主要在团队已有的 Windows、macOS 电脑上开发，无需专用 GPU 或训练大模型。手机采用 Web 交互，中继服务器只承担连接与转发。运行成本主要是模型与语音 API 以及服务器费用，可通过任务预算控制。

系统基于 Node.js、TypeScript 和开源框架实现。团队先打通核心执行链路，再接入跨应用管理、语音和辅助交互，通过自动化测试、持续集成和真实任务记录开展迭代。

三、AI 技术与开源及第三方资源选择

3.1 AI 技术的作用

多模态模型根据自然语言目标、当前截图、历史动作和任务状态理解界面、提取信息并选择操作。项目通过统一 Provider 接口适配 GLM、Qwen、DeepSeek 及火山方舟等

调用渠道；语音识别服务将口头要求转换为任务输入或纠正文本。模型动作由 Runtime 检查并派发，实际效果结合回执、后续画面与最终产物核验。

3.2 开源及第三方资源选型

项目选择开源 **CUA** 连接浏览器与原生应用，复用其截图、输入和窗口控制能力，减少底层驱动重复开发。Windows UI Automation、macOS Accessibility 与受管浏览器 DOM 提供可选结构信息，与视觉操作互补。

React 承担手机界面，Fastify 与 WebSocket 承担电脑服务和通信；TypeScript、Vitest 与 GitHub Actions 支撑构建及验证。模型推理、语音识别使用第三方服务，任务播报利用浏览器和设备系统能力。具体开发资源的版本、许可、使用方式及义务见附后的《开源及第三方资源使用清单》。

3.3 自主开发与权利边界

团队自主实现 Computer Harness 的上下文编译、计划与记忆接入、界面候选选择、跨应用状态管理、执行控制及手机协作，并以 Apache-2.0 开放原创代码。CUA 和其他组件保留各自许可，模型 API 按服务条款调用。开发使用 AI 辅助编程、测试和文档整理，团队负责方案、审查与验证；与维护者共同完成的贡献按实际提交和合并记录说明。

3.4 AI 与开源技术的融合

模型提供语义理解与动作选择，CUA 提供平台操作，自主 Harness 则把两者组织成持续的任务执行：每轮提供适用信息，检查提案是否可执行，再将回执与新观察送回决策。手机、语音和审批连接同一运行状态，使用户介入可以直接影响后续操作。真实使用发现的问题进一步形成复现、测试及上游修复，构成产品开发与开源协作的联系。

四、系统设计与实现

4.1 Agent Harness：把模型决策组织为持续执行的任务

Computer Harness 是模型与桌面工具之间的智能体运行支撑层。CUA 提供截图、输入和窗口控制，模型理解目标并选择动作；团队自主实现的 Harness 负责组织决策所需信息、验证工具调用、维护任务状态及处理用户介入。一次任务对应一个 Run，窗口切换、审批和纠正均在该 Run 内推进，避免把长程任务拆成相互失去状态的模型请求。

例如，用户要求“查车票并发到微信”，模型每一步都需要知道查询条件、当前画面、已经查到的信息以及用户后来补充的要求。Harness 从任务记录中选出这些信息，再交给模型决定下一步。这个过程可概括为：

$$
\begin{aligned}
C_t &= \text{Compile}(G, U_{\le t}, O_t, H_t, P_t, M_t;\ B), \\
(A_t, Z_t) &= \text{Model}(C_t).
\end{aligned}
\tag{1}
$$

第一行表示准备模型输入：G 是目标，U 是后续纠正，O 是当前观察，H 是动作历史，P 和 M 是启用后的计划与记忆，B 限制输入长度。Compile 表示自主实现的信息选择与组织过程。第二行表示模型据此返回下一步动作 A，并描述前一步的结果 Z。动作先经过 Harness 检查，再交给 CUA 执行；模型生成了调用，并不意味着电脑已经完成操作。

模型服务、桌面驱动、上下文、计划和记忆通过组件接口接入，分别处理模型协议、平台操作和任务信息。**模型决定下一步，Harness 决定这一步当前能否执行。**例如模型正在生成发送动作时，用户补充“先不要发”，执行循环会检查这条新要求，使尚未执行的发送动作失效，再组织下一次决策。

4.2 多模态观察与候选控件选择

视觉截图是决策的通用基础。结构采集开启时，Windows UI Automation、macOS Accessibility 和受管浏览器 DOM 进一步提供控件名称、角色、状态及位置。适配器保留元素来源和当前观察的绑定关系；模型可以选择元素引用，也可以根据截图返回坐标。浏览器内容区与浏览器工具栏分别保留 DOM 和原生控件证据，结构候选缺失时仍保留视觉操作路径。

系统先合并重复控件，再选择与当前操作有关的控件。同一个网页按钮可能同时被 DOM 和系统无障碍接口发现，去重时综合比较控件类型、名称和矩形区域的重叠程度。网页内容优先保留 DOM 记录，浏览器工具栏优先保留原生控件记录。随后比较控件文字与当前操作意图、用户纠正、计划和目标的匹配程度，并考虑焦点与可编辑状态。每个候选元素 e 的得分为：

$$
s(e) = w_r I_r(e) + w_l n_l(e) + w_u n_u(e) + w_p n_p(e) + w_g n_g(e) + b(e). \tag{2}
$$

I_r 标记元素是否靠近最近需要重新处理的区域；四个 n 分别统计元素文字与当前意图、用户纠正、计划、目标的匹配数；w 是对应权重，b 是焦点、可编辑性等附加分。当前意图和最新纠正权重较高。例如正在修改出发日期时，“日期”控件应比长期目标中的“微信”更靠前。分数由明确规则计算，用于筛选给模型看的控件，不替代模型判断要点击哪一个。

排序后按恢复区域、结构化控件、焦点或可编辑控件、相关全局候选分配名额；多来源同时存在时约束单一来源占比，再将剩余容量用于补充候选。因此既不会简单截取控件树最前面的若干项，也不会让大型 DOM 列表完全挤掉原生控件。每个入选元素保留选择理由，便于从轨迹核对目标为何被召回或遗漏。

引用选择与实际执行分开验证。模型返回的元素引用由适配器解析为当前控件及动作；几何映射结合截图尺寸、窗口边界与采集来源完成，执行前复查引用和目标状态。

坐标操作使用对应观察的坐标空间。全桌面模式切换应用后仍观察整个桌面；窗口模式则更新绑定目标及局部坐标，防止两种空间被混用。

4.3 上下文组织：让模型知道做过什么、结果如何

长任务不能只靠不断追加历史对话。系统将固定执行协议与工具定义放在稳定前部，把用户纠正、当前截图、运行预算、相关记忆等变化内容放入动态消息。稳定前缀便于利用模型服务的提示缓存，Provider 同时记录服务返回的缓存读取量；具体命中取决于服务端及实际请求内容。

删除旧历史时，把一次调用和它的返回结果作为整体处理。例如“点击查询”与“点击已执行”需要一起保留或一起移出，不能只留下动作而让模型猜测是否执行。系统优先保留目标、用户纠正和最近一次界面操作的完整记录，再为其他内容分配长度。组织模型消息时，先补齐调用对应的结果，再放入后续纠正；原始日志仍按实际时间保存，便于追溯。

当前视觉输入使用最新截图，历史操作主要通过文本回执与状态记录传递。过时的元素目录不能仅因仍存在于历史中继续使用：编译器检查它与当前观察、会话和操作界面的绑定关系。预算不足时，可选目录或指导可被省略；必需输入无法容纳时明确报错，不静默丢掉用户条件。裁剪追踪记录所选事件、各信息块预算及省略原因，使上下文问题可以被定位。

Planning 维护阶段目标、进行状态和阻塞事项；Memory 保存后续仍需使用的事实。两者通过工具调用更新，而非每个鼠标动作都建立一个计划。例如，查询与比较、整理结果、确认并交付可以分别成为阶段。允许组合的计划或记忆更新可与 GUI 动作同轮返回；含副作用动作后的终止需等待回执，在独立轮次完成，避免模型提前宣告任务结束。

4.4 记忆检索：先检查是否适用，再判断是否相关

记住的信息未必一直适用。例如余票会变化，昨天查询的结果也不能直接用于另一天的行程。系统为记忆记录来源、所描述的对象、所属任务或会话，以及稳定、任务相关、短期等类别。检索时先排除范围不符或已被替代的记录，再把需要核实的信息单独列出，最后在各组内按相关程度排序。其筛选规则为：

$$
\begin{aligned}
\mathcal{F}_t &= \{f \in M_t \mid \mathrm{ScopeMatch}(f,t) \\
&\qquad \wedge \mathrm{SubjectValid}(f,t) \wedge \neg\,\mathrm{Superseded}(f)\}, \\
\mathcal{F}_t^{\mathrm{check}} &= \{f \in \mathcal{F}_t \mid \mathrm{NeedsCheck}(f) \vee \mathrm{ShortLived}(f)\}, \\
\mathcal{F}_t^{\mathrm{use}} &= \mathcal{F}_t \setminus \mathcal{F}_t^{\mathrm{check}}.
\end{aligned}
\tag{3}
$$

第一行得到符合任务范围、对象仍适用且未被新记录替代的集合 F。第二行把其中待核实或短期的信息放入 check 组，其余进入 use 组。ScopeMatch 和 SubjectValid 分别检查范

围与对象；Superseded 表示已被替代。这样，即使一条旧余票记录与问题十分相似，也会作为“需重新核实的信息”呈现，而非直接用于回答。

检索查询由显式查询、最新用户纠正、原始目标及近期动作提示构成，优先为显式查询和最新纠正预留长度，因此 Memory 不依赖 Planning 是否开启。系统支持词法检索与可选语义向量检索：具备有效向量时使用余弦相似度，未配置或获取失败时使用词法结果；精确标识匹配获得额外优先级，同分记录按更新时间排序。查询向量和事实向量采用有界缓存，并对请求次数、候选量及等待时间设置上限。

检索过程还检查异步返回期间记忆状态是否变化，避免把旧快照的结果重新送回上下文。最终分别呈现“适用事实”和“需要重新核实的历史信息”，记录候选排除原因及召回来源。例如，车次时刻可作为后续整理依据，余票则更适合作为短期信息；当日期条件改变时，旧查询结果需要重新核对适用性。这里的状态约束由 Harness 实现，事实内容的更新仍由模型工具调用和当前证据共同完成。

4.5 执行闭环、状态评估与受控恢复

运行时把观察、动作提案、真实派发、回执与后续观察分别记录。操作对象使用稳定会话身份，以及 Surface 身份和版本管理；窗口、页面或已确认子界面发生变化时，旧引用失效。Surface Registry 管理当前任务实际使用的操作对象，平台接口提供身份、前台状态和父子关系证据。跨应用目标由模型显式选择，弹窗仅在关系可验证时作为子界面进入与退出。

用下一轮画面检查前一步是否奏效。模型在正常决策中同时报告前一步属于“预期变化、无效果、非预期变化或不确定”，并给出可见依据，无需另发一次评判请求。例如点击查询后仍停留在原页，模型可说明“未出现查询结果”。Monitor 把这段说明与画面变化、重复动作和回执一起检查，生成下一轮提示。仅记录模式保存提示供分析，指导模式才将仍适用的提示送给模型，并限制重复提醒。切换界面或收到纠正后，旧提示需要重新检查。

工具成功与任务成功使用不同证据。输入回执说明底层派发情况，后续画面才用于判断文字是否出现、对话框是否关闭或消息是否送达。对于可能已产生副作用但结果不明的操作，运行时保留未决状态，经重新观察或人工确认处理，不直接重发。输入批处理也限于同一控件中的允许组合，例如全选后输入；每项分别检查，失败或用户纠正后停止未执行的后缀。

4.6 将审批、纠正与无障碍交互纳入同一运行

Risk Guard 在实际派发前检查动作的影响声明与本地规则，区分普通操作和需要用户确认的行为。审批请求绑定具体动作及当前状态，用户同意后仍需复核执行条件。可选语义风险评估通过配置启用；默认 Host 保留本地检查和必要审批，不额外开启风险模型请求。

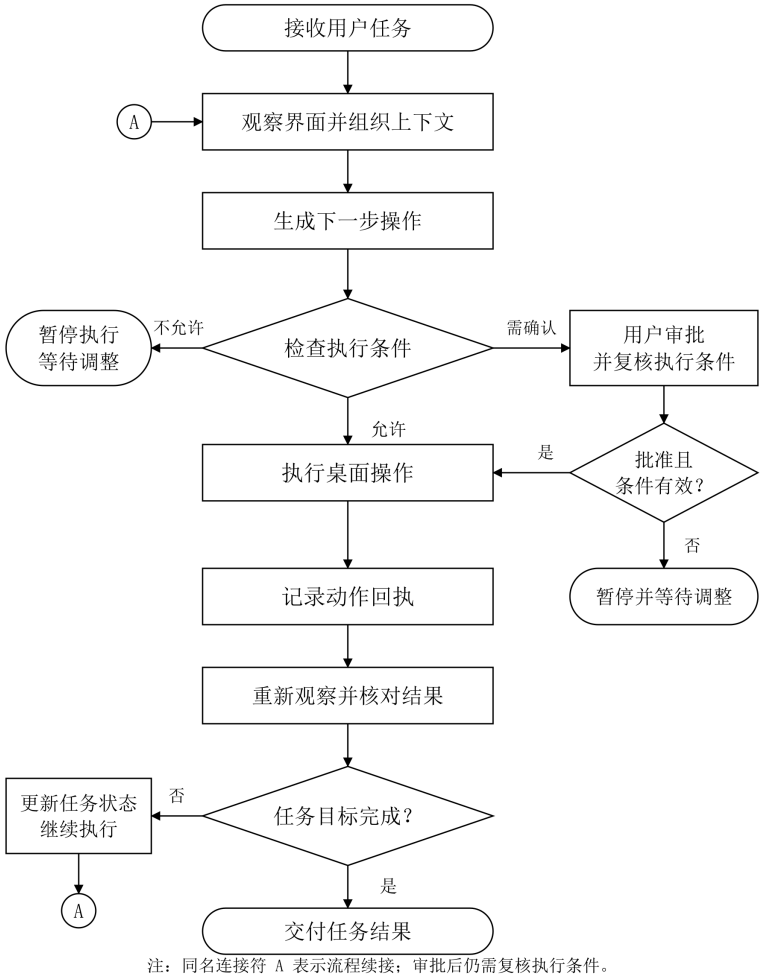


图 2　桌面任务的执行闭环与审批分支。观察和回执共同进入下一轮决策；用户纠正、审批与停止在执行边界生效，任务未完成时沿连接符 A 继续推进。

手机上的补充要求以运行命令进入同一 Run。执行循环在模型返回后、审批等待后及批处理动作之间检查命令；新纠正可以使尚未派发的旧决策失效，随后携带已有任务事实重新决策。该机制保留已完成动作的记录，同时避免执行过时计划。

语音输入转换为与文字相同的目标或纠正；进度播报消费当前观察确认的阶段结果，而非逐个朗读点击意图。手机 Web 为控件提供名称、分组、焦点与状态反馈，运行消息保留完整可读文本，可由系统旁白访问。大字、高对比度、减少动画和回答偏好降低使用负担；个性化偏好作为新任务上下文参与表达，不改变授权范围与审批规则。

4.7 自主实现与工程核验

上述上下文编译、记忆准入与检索、候选选择、状态机及人机控制机制均位于团队自主实现的 Harness 中。模型承担语义理解与动作选择，CUA 承担平台观察与输入。团队通过对应的单元测试和运行轨迹检查过期引用、预算裁剪、纠正使旧调用失效以及部分输入等边界，再以真实任务验证它们如何协同。测试与代码审查同时保留 Windows、macOS 的平台差异，不以某一端的适配结果代替另一端验收。

算法 1 是执行循环的伪代码概括，函数名表示处理步骤。E 保存已发生的事件，S 表示当前任务状态。每轮先处理用户命令、观察界面，再准备模型输入。模型返回后，先检查调用是否合法、是否已被纠正；需要审批的动作等待用户确认，执行前再核对当前目标。FreshCheck 对应这次核对，Superseded 判断旧动作是否已被新要求取代。若只执行了一部分或结果不明，停止剩余动作并重新观察；例如消息是否发出不明确时，不直接再发一次。

算法 1　带状态再验证的 GUI 动作调度

输入： 目标 G，操作范围 W，上下文与运行预算 B
输出： 任务结果 R，事件轨迹 E

```
S, E ← InitRun(G, W, B)
while Active(S) and BudgetAllows(S, B) do
    S, O ← ApplyCommandsAndObserve(E)
    C ← Compile(G, O, E, Plan(S), Memory(S), B)
    A, Z ← Model(C)
    E ← AppendDecision(E, A, Z)
    if Superseded(A, E) or not ValidCallGroup(A) then
        Reject(A); continue
    end if
    for a in A do
        d ← GuardAndResolveApproval(a, S, W)
        if d != ALLOW or Superseded(a, E) or not FreshCheck(a, S) then
            RejectRemaining(A); break
        end if
        r ← Execute(a); E ← Append(E, a, r)
        if r.status in {PARTIAL, UNKNOWN} then
            HoldUnresolved(S, a); RequestObservation(S); break
        end if
        O ← Observe(); E ← Append(E, O)
    end for
    S ← Reduce(E)
end while
R ← Finalize(S); CloseSession(S)
```

4.8 团队分工与过程记录

详细分工见 GitHub 的代码提交与Pull Request 记录。相关记录保留成员贡献、版本迭代、代码审查及问题解决过程。

五、真实场景验证与成果展示

5.1 产品形态与交互

Lightspeaker 将手机作为生活任务的委托与控制入口，电脑负责在真实网站和桌面应用中执行。用户可以输入文字或语音、查看当前进度与实际画面，在需要时补充要求、纠正操作，并确认关键步骤。图 3 展示了实际录屏中的任务页、显示设置、个性化偏好与语音通知设置。



图 3　手机端实际界面节选。（a）目标与任务进度；（b）大字及显示设置；（c）个性化回答偏好；（d）语音通知设置。画面取自团队实录，仅作裁切与排版。

手机端的大字、高对比度、减少动画与系统读屏适配，和任务过程中的文本、语音通知共同构成辅助交互方案。用户可以按自己的阅读习惯调整说明详略与表达方式，相关偏好随新任务进入上下文，参与后续回答与进度表达。

5.2 真实任务测试的组织

团队依据六类生活任务卡，在 Windows 与 macOS 上开展真实环境实验，使用官方网站、桌面应用与授权账户，检查信息查询、条件核对、内容整理、应用切换和结果交付。实验明确目标、起始环境与操作范围，以来源页面、界面截图、动作回执和实际交付核对过程，任务卡与证据随仓库维护。

测试关注三个相互联系的方面：一是模型能否根据用户需求理解页面与组织操作；二是运行时能否在界面变化、跨应用和用户介入时维持任务连续性；三是手机上的进度、审批与最终内容能否与电脑执行过程对应。来源页面、文件内容或应用中的新增消息用于核对实际交付，测试反馈持续用于完善运行控制和交互体验。

5.3 六类生活场景与代表任务

图 4 概括了任务覆盖的六类生活场景。各类别根据日常需要组织具体委托，既包含单一网站的信息整理，也包含需要桌面应用参与的操作与交付。



图 4　六类真实生活任务覆盖及代表委托。类别划分用于组织真实任务测试。

政务任务形成敬老卡使用说明、居住证签注说明和面向长辈的津贴提示文档；就医任务整理院区、科室位置与到院准备，并将交通信息交付微信。购物任务将商品比较、预算筛选和退货说明保存为文稿；出行任务完成车次与路线比较，以及微信消息交付。缴费任务依据官方价目表整理电价、核算演示账单；个人事务任务将通知整理为待办，调整冲突安排，并修改、保存及重开计划副本。各类交付分别以来源、正文内容、文件或新增消息核对，详见随附佐证材料。

5.4 代表 Demo：跨应用执行与用户介入

以下两个案例分别来自 **Windows** 与 **macOS（Mac）**的真实运行，展示不同桌面系统上的任务执行与用户介入。

5.4.1 Windows：查车次并通过微信交付

用户通过手机委托查询指定日期上海至雁荡山、上午 10 点至 12 点出发的直达列车，整理车站、时刻、历时、二等座票价和余票，再发送至微信“文件传输助手”。电脑在携程查询并整理信息，随后切换到微信；用户在手机确认发送操作后，系统完成消息交付并反馈结果。



图 5　同一次 Windows 跨应用任务的实际执行画面。（a）携程车次查询；（b）手机发送审批；（c）微信中的结果消息。三幅画面按查询、确认、交付三个阶段截取自同一次任务录屏。

Planning 保留查询与交付阶段，上下文携带查询条件和已获得的信息，使微信操作接续同一任务；审批将发送动作与用户确认关联，执行后再观察新增消息。视频保留真实执行与任务播报，标识等待变速及产品旁白。

完整过程见随附的 Lightspeaker 项目介绍视频，时长约 4 分 30 秒。

5.4.2 macOS：商品查询、手机纠偏与桌面报告交付

用户通过手机委托比较 MacBook Air 的两套配置，并将选购建议写入已打开的文本编辑文稿、保存至桌面。电脑在 Apple 中国官网查询配置与价格；用户查看执行情况后，在手机端输入纠偏要求“采用这个配置即可开始完成报告”。系统接收要求，使尚未执行的旧滚动操作失效，随后在同一个任务中转入写稿和保存，最终保持已保存文稿打开。



图 6　同一次 macOS 任务的录屏节选。（a）官网查询；（b）手机补充要求；（c）文稿与桌面文件。画面按查询、纠偏、交付阶段排列，手机输入法组合状态保留原貌。

用户纠正进入当前 Run 后，执行循环检查尚未执行的决定，Planning 更新后续安排，文稿保存后的画面用于核对交付。商品预算调整和计划副本修改等任务进一步采用这一机制，将新的要求落实到最终文稿，无需取消后重新委托。

5.5 工程验证与迭代

真实任务测试与工程验证共同推进系统开发。项目通过自动化测试检查工具协议、任务状态、上下文组织、模块组合、资源清理和 Host/Relay 通信，通过真实模型接口与原生应用测试核对调用、窗口和输入行为，并在 Windows 与 macOS 环境开展平台适配验证。

GitHub Actions 承接构建与回归检查，代码审查和实际使用反馈用于完善跨应用操作、用户纠正、审批及手机交互。开发中发现的底层输入问题进一步形成独立复现和测试夹具，与 CUA 上游维护者的协作相衔接，使产品迭代和开源基础改进形成持续联系。

六、创新性与开放价值

6.1 面向真实任务的系统创新

Lightspeaker 的创新体现在开源技术的系统化应用与面向真实用户的交互设计两个方面。

在技术层面，项目基于多模态模型和开源 CUA，自主开发 Computer Harness 运行时，将跨应用状态管理、执行结果核对、任务上下文和用户控制整合为统一的执行机制。相较于直接连接模型与桌面操作工具的基础方案，Computer Harness 能够持续管理当前操作对象，在界面变化后重新确认引用，区分动作回执与实际结果，并在失败或结果不确定时采取受控恢复。统一的模块接口还支持替换模型、桌面适配和运行策略，为不同任务中的能力复用提供基础。

在应用层面，Lightspeaker 将手机交互与实际桌面执行连接起来。用户通过文字或语音提出需求，在手机上查看进度、补充要求和确认关键操作。系统还提供大字、读屏支持、进展播报及个性化配置，关注老年人和数字工具使用门槛较高的群体。上述交互直接参与任务运行，使用户能够以熟悉的方式使用智能体，同时保留对关键操作的决定权。

6.2 开放成果与复用价值

项目原创代码采用 Apache-2.0 许可证，项目成果包括 Computer Harness 运行时、桌面适配、手机协同控制、任务协议、测试工具及技术文档。

开发者可以基于统一接口替换模型服务、调整任务规划与记忆策略，或接入其他桌面环境；也可以利用已有测试和运行记录分析智能体执行过程。项目提供的真实任务案例、问题复现和回归测试，为相关工具的开发和验证提供了参考。

上述成果通过 GitHub 仓库维护，第三方组件分别遵循其许可证与服务条款，开放范围以仓库中的源码及授权声明为准。

6.3 与开源社区维护者共同解决实际问题

在 Lightspeaker 的开发过程中，团队发现 Windows 现代记事本存在多行输入异常：模型生成的内容正确，但目标应用在换行后可能出现字符丢失或错误。团队将问题从完整任务中提取，建立独立复现，并为 WPF、WinUI3 编写测试夹具，通过应用实际接收的文本验证输入结果。

这些测试帮助团队与 CUA 维护者共同定位前台焦点恢复和输入队列处理之间的时序问题。维护者进一步完善修复方案，相关改进已通过 CUA PR #4500 合并 [2]。团队贡献了问题复现、测试夹具和回归验证，使实际使用中发现的问题转化为上游可持续维护的测试资产。

双方随后继续合作解决文稿输入的可靠性问题。团队在 PR #4836 中提出基于控件能力的前台 Document 输入方案 [3]，上游维护者在 PR #5587 中保留团队原始提交与作者署名，并进一步整合文本读回验证及不确定输入的重复执行防护 [4]。该 PR 于 2026 年 10 月 21 日合并。

此外，团队向 OpenClaw 社区贡献的 PR #126399[5] 和 PR #127177[6] 均已合并，分别解决 Computer 任务结束后的资源复用问题，以及 Windows 不同客户端切换时的重复

设备审批问题。OpenClaw 属于团队在深入 computer-use 开源生态中参与贡献的独立开源项目。

团队还参与通义 OpenGCUA 的 Windows 支持协作 [7]。在维护者确认方向后，团队公开了 SDK 与 CUA 的接口映射、控件能力及输入语义分析，并记录浏览器、Excel 和 Word 的初步实验。设计保持公共 SDK 与任务运行语义一致，由 Windows 适配层处理平台差异，探索结构化控件操作与程序化执行的结合，减少多步任务中的重复模型请求。

Computer-Harness 也被收录进 XERJ Corpus Hub[8]。平台以固定源码版本提供可检索的实现参考，收录团队成员对权限保留和模型服务预检的提交，保留作者、仓库和 Apache-2.0 许可信息，供开发者与编程智能体检索复用。

6.4 后续维护与社区协作

项目将持续通过 GitHub 维护代码、文档与问题反馈，完善安装指南、平台兼容性及真实任务测试。后续重点包括改进跨应用执行稳定性、补充可复现的评测案例、完善 Windows 与 macOS 的使用流程，并吸收不同用户群体的体验反馈。

团队将继续跟进 CUA 与 OpenGCUA，通过测试、Issue 和 Pull Request 参与社区协作。

七、安全、隐私与合规

7.1 开源许可与知识产权

Lightspeaker 原创代码采用 Apache-2.0 许可证。项目使用的 CUA、前后端框架及其他第三方组件分别遵循其适用许可证，模型与语音识别服务按照相应服务条款调用。项目通过 LICENSE、THIRD_PARTY_NOTICES 和独立资源清单记录第三方资源的来源、版本、使用方式及许可要求，并根据实际分发内容履行署名与声明义务。

团队自主实现、第三方依赖及上游协作贡献分别说明。成员和共同权利人的贡献及授权范围按实际记录核查，不将第三方软件、服务或素材纳入项目原创代码的授权范围。

7.2 数据使用与隐私保护

Lightspeaker 在用户授权的电脑和账户环境中执行任务。模型凭据及受管浏览器配置由电脑端 Host 管理，手机通过配对参与任务。模型请求可能包含任务文本、界面截图与必要上下文，语音识别则使用相应第三方服务。

系统支持选择窗口或全桌面操作范围，并通过配对鉴权、设备撤销和访问限制管理远程控制。公开仓库和演示材料使用测试资料及脱敏记录，不公开真实密钥、个人登录状态或无权披露的聊天内容。对于访问到的第三方网站信息，按其实际使用授权处理，不将公开可访问内容直接视为可自由训练、改编或再分发的数据。

7.3 AI 输出与操作安全

多模态模型可能出现界面识别偏差、信息遗漏、错误操作或不准确的完成声明，网页内容也可能包含误导模型的指令。Computer Harness 通过工具参数校验、操作范围限制、执行回执、重新观察及风险审批等机制控制执行过程，并将外部页面内容与系统授权规则分开处理。

对于信息查询任务，项目通过来源页面核对关键事实；对于文件编辑和消息发送等任务，结合运行记录及实际交付结果进行验证。涉及敏感修改或对外提交的操作由相应风险策略与用户确认共同控制。

团队在开发过程中使用生成式 AI 辅助编程、测试和文档整理，并对生成内容进行人工审查、实际验证及必要的来源和权利核查，遵守所使用工具与服务的适用条款。

演示视频使用真实手机与电脑录屏，产品介绍旁白由豆包语音合成模型 2.0 的预置 Vivi 音色生成，解说文案、画面对应关系及最终剪辑由团队审核。AI 旁白与系统实际播报在画面中分别标识，片尾说明声音来源；未使用真人声音复刻。

报告中的六类生活场景示意图使用 AI 辅助绘制，用于说明任务分类；产品界面与跨应用案例截图取自真实录屏。

7.4 适用范围与技术限制

系统面向授权的日常桌面任务，执行效果受到模型、网站变化、网络与平台差异影响。涉及资金及账户安全的操作保留人工确认。读屏支持文字、控件及任务状态访问；实际桌面截图仍需视觉核对或协助。适老化及无障碍体验将结合真实用户试用持续改进。

八、总结与展望

Lightspeaker 以自主 Computer Harness 连接手机交互、模型决策与真实桌面执行，将跨应用状态、执行核对、用户纠正和风险审批纳入统一任务过程。项目围绕真实生活需求开展实验，以语音和无障碍交互降低操作门槛，并通过 CUA 修复、OpenGCUA 协作及源码开放参与开源建设。

后续将完善执行效率与平台适配，开展定量评测及更多真实用户试用，改进部署和文档，使系统更便于在日常生活中使用。

九、附录

9.1 参考资料

[1] trycua. CUA 开源项目 [EB/OL]. https://github.com/trycua/cua.

[2] trycua/cua. PR #4500: fix(cua-driver): drain Windows foreground input before restoring focus[EB/OL]. https://github.com/trycua/cua/pull/4500.

[3] trycua/cua. 团队前台 Document 输入方案，PR #4836[EB/OL]. https://github.com/trycua/cua/pull/4836.

[4] trycua/cua. PR #5587: fix(cua-driver): verify foreground Document text insertion[EB/OL]. https://github.com/trycua/cua/pull/5587.

[5] openclaw/openclaw. PR #126399: fix(codex): allow computer reuse across completed runs[EB/OL]. https://github.com/openclaw/openclaw/pull/126399.

[6] openclaw/openclaw. PR #127177: fix(gateway): prevent repeated Windows device approvals[EB/OL]. https://github.com/openclaw/openclaw/pull/127177.

[7] Tongyi-MAI/OpenGCUA. Windows 支持设计与 SDK 映射，Issue #6[EB/OL]. https://github.com/Tongyi-MAI/OpenGCUA/issues/6.

[8] XERJ Corpus Hub. Computer-Harness 源码收录与团队成员贡献记录 [EB/OL]. 贡献记录与源码语料.

9.2 提交材料与开源入口

随作品提交本技术报告、附后的《开源及第三方资源使用清单》及 Lightspeaker 项目介绍视频。佐证材料汇集开源贡献、双平台案例、六类任务卡及公开证据入口。视频展示产品交互、Windows 查车次与微信交付及开源贡献。

项目成果链接：Computer-Harness 开源仓库。

上游协作和源码开放记录见参考资料 [2]—[8]。

开源及第三方资源使用清单

序号	资源名称及类型	版本/来源	许可证/授权类型	使用/开放方式	关键义务/限制	团队自主修改或开发内容	合规状态
1	CUA（开源桌面操作工具）	SDK 0.22.2；trycua/cua	SDK、daemon：MIT；原生 binding：MIT AND MPL-2.0	调用截图、输入和窗口能力；驱动单独安装	保留版权、许可及原生组件声明；分发修改组件时履行相应源码义务	自研 Computer Harness、跨应用管理与执行核验；参与上游输入问题复现、测试及修复	已核验
2	React / React DOM（界面框架）	19.3.0；React	MIT	构建手机 Web 界面	分发时保留版权和许可声明	任务控制、审批、纠正、个性化与无障碍交互	已核验
3	Fastify、ws（服务与通信组件）	5.12.5、8.21.3；Fastify、ws	MIT	电脑端服务及手机中继通信	保留版权和许可声明	配对鉴权、连接管理、任务状态与消息协议	已核验
4	TypeScript、Vite、Vitest（开发工具）	5.7.2、8.3.1、2.1.8；TypeScript、Vite、Vitest	Apache-2.0、MIT	编译、前端构建与自动化测试	保留适用许可和声明	自主系统代码、构建配置与回归测试	已核验
5	GLM、Qwen、DeepSeek（模型服务）	智谱、阿里云、DeepSeek、火山方舟	服务商 API 条款	按用户配置调用云端模型，不分发模型权重	遵守服务条款；保护密钥及任务数据	模型适配、上下文组织、规划记忆及工具执行协议	已核验
6	Qwen 实时语音识别、手机系统语音（语音服务）	阿里云服务及用户设备浏览器接口	服务条款、系统使用许可	语音输入与任务关键进展朗读	告知录音用途；不分发系统音色	语音输入接入、播报调度、与系统读屏协调	已核验